EXP 16: Compare different types Classification Algorithms and evaluate their performance.

CODING:

```
# Comparison of Classification Algorithms

# Sample dataset
X = [
   [1, 1], [2, 1], [1, 2],
   [6, 6], [7, 6], [6, 7]
]

Y = ["A", "A", "A", "B", "B", "B"]

# Test data
test = [[2, 2], [7, 7]]
actual = ["A", "B"]


# 1. KNN Classifier
def knn(point, k=3):
   distances = []

   for i in range(len(X)):
      d = ((point[0] - X[i][0]) ** 2 +
         (point[1] - X[i][1]) ** 2) ** 0.5
      distances.append((d, Y[i]))
```

```
    distances.sort()
    labels = [item[1] for item in distances[:k]]

    return max(set(labels), key=labels.count)


# 2. Naive Bayes Classifier
def naive_bayes(point):
    classes = set(Y)
    scores = {}

    for c in classes:
        rows = [X[i] for i in range(len(X)) if Y[i] == c]
        score = len(rows) / len(X)

        for j in range(2):
            mean = sum(row[j] for row in rows) / len(rows)
            variance = sum((row[j] - mean) ** 2
                           for row in rows) / len(rows)

            if variance == 0:
                variance = 0.0001

            import math
            probability = (1 / (2 * math.pi * variance) ** 0.5) * \
                math.exp(-(point[j] - mean) ** 2 / (2 * variance))

            score *= probability
```

```
        scores[c] = score

    return max(scores, key=scores.get)



# 3. Decision Tree (Simple Decision Rule)
def decision_tree(point):
    if point[0] < 4:
        return "A"
    else:
        return "B"



# Evaluate performance
algorithms = {
    "KNN": knn,
    "Naive Bayes": naive_bayes,
    "Decision Tree": decision_tree
}

for name, algorithm in algorithms.items():
    predictions = [algorithm(point) for point in test]

    correct = sum(predictions[i] == actual[i]
                  for i in range(len(actual)))

    accuracy = (correct / len(actual)) * 100
```

```
print("\nAlgorithm:", name)
print("Predictions:", predictions)
print("Accuracy:", accuracy, "%")
```

OUTPUT:

